

PaperBlast

Implementation Audit Report

Original Product & Implementation Plan vs. Current Research_Blast_Radius-main Codebase

Audit date: 5 September 2026

Source specification: "PaperBlast — Final Product Definition, Refinement & Production Implementation Plan", dated 21 August 2026.

Audited artifact: Research_Blast_Radius-main.zip

Audit purpose. This report compares the requirements in the supplied 26-page PDF against the uploaded codebase. It focuses on whether requirements are actually implemented in the runtime path, not merely represented by schemas, documentation, names, or UI placeholders.

Important scope note. Line numbers in this report are exact line numbers from the uploaded ZIP snapshot. They are snapshot-specific and will change if files are edited. Findings marked as "implemented" mean the code evidence supports the requirement; "partial" means the concept exists but does not yet satisfy the production contract; "violation" means the feature exists in a form that conflicts with the PDF.

1. Executive Audit Summary

The updated repository contains meaningful architectural work: a Python AST parser, evidence-oriented domain models, verification statuses, manuscript research objects, provenance graph code, Research PR/authentication structures, CI/test files, and an EvidencePanel. These changes address several deficiencies identified in the PDF’s original codebase audit.

However, the central production workflow is not yet compliant with the final blueprint. The runtime still contains a simulated analysis job, a static/heuristic fallback path, an in-memory database, AI-supplied lineage, a researcher-facing Blast Radius Score, a CLI that explicitly sends mock payloads, and a deployment adapter that points at a missing Node server entry point while the actual backend is FastAPI.

Area	PDF expectation	Current state	Assessment
Core impact engine	Deterministic evidence → verification → finding	Static keyword matching + AI synthesis; AI can supply lineage	BLOCKER
Python analysis	Real AST/data-flow analysis	Real AST extraction, but regex fallback on syntax failure and limited relationship modeling	PARTIAL
Evidence	Every important finding has navigable evidence	Evidence domain + UI panel exist; AI can still contribute factual-looking lineage	PARTIAL
No fabrication	No silent fallbacks / no result after required failure	Static fallback and simulated jobs remain	VIOLATION
Persistence	Persistent Research Project/versioning	In-memory adapter only	MISSING
Research PR	Real commit/diff + impact + review	PR object/review APIs exist; not connected to real Git diff analysis	PARTIAL
What-if AI	AI interprets; deterministic engine determines facts	AI still drives section/lineage synthesis	PARTIAL
Researcher UI	Simple workspace; technical diagnostics in developer mode	AST/agent/blast-radius UI remains prominent	VIOLATION
Testing	Unit + integration + mutation + FP/FN + security + E2E	Several unit tests; test collection currently fails	BLOCKER
Deployment	Reproducible production architecture	Vercel points to api/index.js → missing server/index.js; actual backend is server/main.py	BLOCKER

Overall conclusion: the codebase is a materially improved prototype, but it does not yet meet the PDF’s Production Definition of Done. The highest-risk gaps are not cosmetic: they affect truthfulness, provenance, runtime correctness, persistence, and the ability to prove that a reported impact came from real project evidence.

2. Audit Method and Source Baseline

The PDF establishes the north-star question: if code, parameters, algorithms, configuration, or computational steps change, which experiments, results, figures, tables, claims, equations, and paper sections could be affected — and which appear unaffected? It also defines an explicit accuracy boundary: PaperBlast must not present an unverified inference as a verified fact, invent evidence, or fabricate numerical outcomes. (PDF pp. 2–4.)

The PDF’s implementation blueprint is organized around five critical ideas: deterministic analysis, evidence and verification, persistent research state, researcher-first UX, and lightweight collaboration. (PDF pp. 8–16.) The audit therefore evaluates both presence and runtime semantics.

Repository inspection included source inventory, exact line-number inspection, Python bytecode-independent source review, test execution, and Python compile validation. No external web research was used for the codebase findings.

Validation performed	Result
Repository inventory	75 non-cache files under client/server plus project docs; Python and React/FastAPI paths coexist.
Python compile check	PASS — server and cli.py compile successfully.
Pytest execution	FAIL at collection — 4 modules fail because groq is unavailable in the audit environment.
Frontend dependency/build check	Not executed because node_modules is not included in the ZIP; package.json was inspected.
Deployment wiring inspection	FAIL/RISK — Vercel routes to api/index.js, which imports missing server/index.js.

3. Requirement-by-Requirement Compliance Matrix

ID	PDF requirement / source	Current implementation	Exact code evidence	Assessment
R1	Researcher-first product; change-impact intelligence is primary. PDF pp. 2, 4, 10.	Core analysis remains branded and described as Blast Radius / multi-agent architecture.	client/src/App.jsx:2-6 imports technical viewers; client/src/components/BlastRadiusHeader.jsx:115-120 renders Blast Radius Score; WhatIfConsole.jsx:136-157 uses Blast Radius terminology.	VIOLATION
R2	Replace regex-only Python parsing with real AST. PDF p. 8.	ast.parse + NodeVisitor implemented.	server/engine/code_parser.py:8-102 defines PythonCodeVisitor; 128-134 calls ast.parse and visits tree.	IMPLEMENTED
R3	No fallback that converts parser failure into plausible output. PDF pp. 7, 14.	AST SyntaxError falls back to legacy regex parser.	server/engine/code_parser.py:135-139 calls parse_python_regex_fallback; 144 onward implements regex parser.	VIOLATION
R4	Follow calls, assignments, parameter/data flow; experiment relationships. PDF p. 8.	Graph builder has heuristic relationships, including keyword overlap and AI-suggested edges.	server/engine/graph_builder.py:43-78; especially keyword overlap, symbol-in-section, AI-suggested result→metric edges.	PARTIAL
R5	Claims, parameters, metrics, datasets, equations, tables, figures are first-class manuscript objects. PDF p. 8.	Several extraction routines exist, mostly regex/heuristic.	server/engine/paper_parser.py:242-307 extracts numerical claims, claims, datasets, metrics; earlier routines extract tables/figures/equations.	PARTIAL
R6	Real PDF/DOCX extraction; preserve extraction status. PDF pp. 8, 22.	Frontend accepts PDF/DOCX, but Python extraction is string/UTF-8 based rather than real document parsing.	client/src/components/WhatIfConsole.jsx:115-124 accepts .pdf/.docx; server/main.py:181-192 calls extract_text_from_document; package.json:15-23 has JS parsers but runtime is Python.	VIOLATION
R7	Evidence engine; every important finding has source/location/value/relationship/status. PDF p. 9.	Evidence models/factories and EvidencePanel exist.	server/domain/models.py; server/domain/factories.py; client/src/components/EvidencePanel.jsx:1-96.	IMPLEMENTED / PARTIAL
R8	AI must not be source of truth; deterministic tools collect evidence, verifier validates. PDF pp. 15, 26.	AI receives sections/symbols and returns affected sections and lineage; lineage can be copied into result.	server/engine/impact_engine.py:360-440 prompts for affected_sections/lineage; 532 onward validates AI sections; 553-560 builds lineage from AI result when supplied.	VIOLATION
R9	Explicit statuses; no generic confidence percentages. PDF pp. 12-13.	VerificationStatus exists, but Blast Radius Score/impact score remains.	server/engine/impact_engine.py:312-314; client/src/components/BlastRadiusHeader.jsx:115-120; prompt at impact_engine.py:369,374 asks for overall_impact_score.	VIOLATION
R10	No static/demo findings or silent fallback. PDF p. 14.	Analysis-failed result still contains static lineage and labels engine mode static_fallback; job endpoint simulates completion.	server/engine/impact_engine.py:279-340; server/main.py:33-46.	VIOLATION
R11	Persistent Research Project/versioning. PDF pp. 8-12.	ResearchProject/AnalysisVersion domain exists; persistence provider is in-memory.	server/engine/persistence/db_adapter.py:24-35 and 88-95; Research models in server/domain/models.py.	PARTIAL
R12	Research PR linked to commits/diffs and impact findings. PDF pp. 9, 16.	ResearchPR CRUD/review objects exist, but API accepts version IDs rather than retrieving real Git diffs and running impact analysis.	server/routers/pr_router.py:12-40; 73-106.	PARTIAL
R13	Real-time means real job state, not fake immediate completion. PDF p. 14.	Background job only sleeps 1 second then returns “Analysis Complete”.	server/main.py:33-46, especially line 40 and line 41.	VIOLATION
R14	Security: isolated workers for repository execution; limits; no secrets in prompts/logs. PDF p. 18.	Some ingestion limits/auth exist; no isolated execution worker is implemented.	server/main.py:116-179 clones/extracts repository; no container/job worker isolation; server/engine/jobs.py is in-memory.	PARTIAL
R15	Testing: parser/dependency/evidence/integration/mutation/FP/FN/regression/security/E2E. PDF p. 19.	Eight test modules exist, but collection fails and advanced required layers are not evidenced.	server/tests/*.py; pytest result: 4 collection errors due to missing groq.	BLOCKER
R16	Open-source docs/CI/examples/security. PDF p. 17.	README, architecture, contributing, security, benchmarks and CI are present; some required release artifacts are absent.	Repository root docs; .github workflow exists if present in repository tree; no CODE_OF_CONDUCT.md or CHANGELOG.md in inventory.	PARTIAL
R17	Self-hosting/deployment must be reproducible. PDF pp. 17, 23-24.	Vercel config routes to JS adapter while runtime backend is Python.	vercel.json:1-14; api/index.js:1; server/main.py:31.	BLOCKER

4. Code Analysis & Dependency Model

The PDF requires a Python-first real syntax/data-flow layer and exact source locations. The AST parser is a real improvement, but the relationship engine remains mostly heuristic. This means the system can identify syntax objects without yet proving scientific dependency chains.

A. AST parsing — `server/engine/code_parser.py:8-102, 105-134`

The NodeVisitor records imports, classes, functions, arguments, assignments and calls with file/line/column/source text. This satisfies the structural AST extraction portion.

B. AST failure fallback — `server/engine/code_parser.py:135-139; 144-210`

A SyntaxError invokes `parse_python_regex_fallback`. This conflicts directly with the PDF's no-silent-fallback requirement. Replace this with an explicit extraction failure record and prevent downstream findings from treating the degraded parse as equivalent to a successful parse.

C. Dependency graph — `server/engine/graph_builder.py:43-78`

Relationships include parameter reference, keyword overlap, symbol appears in section, AI-suggested result→metric, and inferred equation variable relationships. The PDF calls for calls, assignments, parameter/data flow and experiment relationships. A real graph needs edges derived from AST references and assignments rather than textual overlap.

D. Change comparison — `server/engine/impact_engine.py:22 onward; compare_graphs`

A `compare_graphs` path exists, but it operates on domain graph objects; the public analysis endpoint still accepts `codeSymbols + paperAST + query` rather than a real base/proposed repository diff.

5. Manuscript / Paper Analysis

The PDF requires claims, parameters, equations, tables, figures, metrics and numerical results to be modeled with exact source locations, and extraction quality must be explicit. The current parser has useful extraction routines but is not a real PDF/DOCX ingestion pipeline.

A. Claims and numerical results — `server/engine/paper_parser.py:242-286`

Numerical claims and claim types are extracted with regular expressions and sentence heuristics. This is a useful prototype, but not enough to reliably preserve page/paragraph/table coordinates or distinguish scientific claim provenance.

B. Dataset/metric extraction — `server/engine/paper_parser.py:288-307`

Dataset names are a hard-coded vocabulary and metrics are a hard-coded vocabulary. This creates false-negative risk for arbitrary research projects.

C. Frontend document support — `client/src/components/WhatIfConsole.jsx:115-124`

UI advertises PDF/DOCX upload. Backend `server/main.py:181-192` routes the payload into `extract_text_from_document`, but the runtime backend lacks the JS pdf-parse/mammoth implementation advertised by `package.json`.

D. Required change — `server/engine/paper_parser.py + server/main.py:whole parser boundary`

Implement real PDF/DOCX/LaTeX adapters returning a common extraction result with success/degraded/failed status, source page/paragraph/table locations, and raw extraction evidence. Do not return raw strings after parser failure as if extraction succeeded.

6. Evidence, Verification & No-Fabrication

This is the highest-priority trust area in the PDF. The repository has made meaningful progress with evidence models and verification statuses, but the actual impact path still permits AI-generated relationships and static

fallback results.

A. Evidence domain — `server/domain/models.py`;
`server/domain/factories.py:domain model + factories`

Evidence, findings and verification status are represented as durable-shaped domain objects. This is aligned with the PDF.

B. AI lineage problem — `server/engine/impact_engine.py:532-560`

The result consumes `ai_result['lineage_graph']` when present. This means the model can effectively author factual-looking edges. The fix is to make AI return only semantic candidate mappings/intents; every displayed lineage edge must be generated from deterministic artifact IDs/source locations.

C. Failure result problem — `server/engine/impact_engine.py:279-340`

`build_analysis_failed_result` creates static evidence/lineage and returns status `ANALYSIS_FAILED`. Even though `affected_sections` is empty, lineage still looks like an analysis result. The PDF says no result should be shown after required processing failure.

D. Required verification gate — `server/engine/impact_engine.py + domain`
`factories:analysis pipeline`

Introduce a strict verifier stage: a finding can become `VERIFIED` only if all referenced artifact IDs, source locations and relationship edges exist in deterministic evidence. AI-originated claims must remain `NEEDS_REVIEW`.

7. Impact Engine & What-If AI

The PDF defines AI as semantic assistance, not authority. The current impact engine is still monolithic around an AI synthesis step and static matcher.

A. Static matcher — `server/engine/impact_engine.py:354` and related helpers

`match_symbols_to_paper` runs before the AI and creates keyword/symbol overlap candidates. This is useful as candidate generation, but keyword overlap must never be treated as dependency proof.

B. AI prompt — `server/engine/impact_engine.py:360-440`

The prompt asks the model for `overall_impact_score`, `risk_level`, `affected_sections` and `lineage_graph`. The PDF specifically says AI should interpret changes and explain evidence, not create factual evidence or force section impact.

C. Required redesign — `server/engine/impact_engine.py:replace`
`calculate_blast_radius flow`

Split into: parse intent → resolve project artifact candidates → deterministic dependency traversal → collect evidence → verifier → optional AI explanation. AI should not choose the final artifact graph.

D. Numerical outcomes — `server/engine/impact_engine.py:369-375`; `result schema`

The prompt still asks for a 0-100 overall impact score. Remove this from the researcher-facing model unless a separately calibrated benchmark supports its scientific interpretation. Exact new metrics must remain “cannot determine without rerunning” until actual execution is available.

8. Persistence, Versioning & Research PR

The PDF requires persistent `ResearchProject` state, `commit/paper` versioning, research artifacts, findings and Research PR objects. The current code has much of the domain vocabulary but stores state in process memory.

A. In-memory database — `server/engine/persistence/db_adapter.py:24-35`,
`88-95`

The production provider is always `InMemoryAdapter`. The file itself states it must be swapped for Postgres.

B. PR creation — `server/routers/pr_router.py:12-40`

A ResearchPR object is created from `base_version_id` and `proposed_version_id`. There is no real Git branch/commit retrieval in this route and no impact report attachment.

C. Review/comments — `server/routers/pr_router.py:42-106`

Comments and review state exist and role checks are present. This is useful collaboration infrastructure.

D. Required implementation — `server/engine/persistence/db_adapter.py` + `server/routers/pr_router.py` + Git connector:production path

Add a real DB adapter, persist

ResearchProject/ResearchArtifact/AnalysisVersion/Evidence/ImpactFinding/ResearchPR, retrieve real commits/diffs, run analysis against the proposed version, attach findings, and enforce stale-version checks.

9. Researcher-Facing Frontend

The PDF says the normal researcher should see a workspace organized around Project Overview, Changes/PRs, Paper Impact, Experiments/Results and Issues. Technical AST/agent terminology belongs in developer mode.

A. Technical navigation — `client/src/App.jsx:1-6` and `270-386`

App imports CodeGraphViewer and DependencyFlow and renders lineage/agent-trace areas. This keeps backend architecture in the normal workflow.

B. Blast Radius Score — `client/src/components/BlastRadiusHeader.jsx:114-120`

The UI explicitly renders “Blast Radius Score” and a percentage. Remove it and replace it with status buckets and evidence summaries.

C. Agent branding — `client/src/components/BlastRadiusHeader.jsx:analysis status text in header`; UI copy

The UI continues to describe multi-agent calculation. Remove agent names from the normal researcher experience; retain diagnostics only in an explicit developer mode.

D. What-if wording — `client/src/components/WhatIfConsole.jsx:132-157`

Uses “Blast Radius” language and presents the action as “Calculate Blast Radius.” Rename to an evidence-backed impact assessment and expose inability-to-verify states clearly.

10. Jobs, Real-Time Processing & Deployment

The PDF’s real-time contract means genuine processing state. The current job endpoint is a placeholder and the deployment wiring is inconsistent.

A. Simulated job — `server/main.py:33-46`

The background task executes `asyncio.sleep(1)` and then returns `{'message': 'Analysis Complete'}`. This is a direct violation of the no-mock production contract.

B. Job storage — `server/engine/jobs.py:whole file`

Job records are process-memory state, so jobs are not durable across instances/restarts.

C. Vercel adapter — `vercel.json:1-14`

Vercel sends `/api/*` to `/api/index.js`.

D. Missing runtime target — `api/index.js:1`

`api/index.js` imports `../server/index.js`. The uploaded repository has `server/main.py` but no `server/index.js`. This is a concrete deployment blocker.

E. Required implementation — `server/main.py` + `server/engine/jobs.py` + deployment config:production path

Use a real worker/job backend, persist job state, run the actual analysis pipeline, expose processing/failure/partial states, and align Vercel/self-host deployment with the actual FastAPI application.

11. Exact Change Plan — File-by-File

The following table is intended to be handed directly to an engineer. It names the current location to modify, what is wrong, and what the implementation should become to satisfy the PDF.

Priority	Current file / lines	Change required	Acceptance condition
P0	server/main.py:33-46	Delete simulated sleep/"Analysis Complete". Call the actual analysis pipeline and persist real job state.	A job cannot reach READY unless parsing, dependency analysis, evidence and verification actually ran.
P0	api/index.js:1; vercel.json:1-14	Replace broken Node adapter or provide a real server/index.js bridge; align deployment with FastAPI.	Clean deployment starts the same backend represented by server/main.py.
P0	server/engine/persistence/db_adapter.py:24-35, 88-95	Implement Postgres/Supabase adapter and make it the production provider; remove unconditional InMemoryAdapter selection.	Research projects, versions, PRs, evidence and jobs survive process restart.
P0	server/engine/impact_engine.py:342 onward	Rewrite into staged deterministic-first pipeline; AI only interprets/normalizes candidate concepts.	Final factual edges/findings are generated from project evidence, not model-created lineage.
P0	server/engine/impact_engine.py:553-560	Do not accept ai_result['lineage_graph'] as evidence. Replace with deterministic graph edges keyed by artifact IDs.	Every displayed edge resolves to deterministic source/target artifacts and locations.
P0	server/engine/impact_engine.py:279-340	Failure result must not contain analysis lineage/evidence that looks like a finding. Return explicit failure diagnostics only.	ANALYSIS_FAILED contains no impact conclusion.
P0	server/engine/code_parser.py:135-139	Remove regex fallback after AST SyntaxError. Record extraction failure/degraded status and block verified findings downstream.	Malformed Python never becomes a plausible parsed project state.
P0	server/engine/graph_builder.py:43-78	Replace keyword-overlap proof with AST reference/call/assignment/data-flow edges; add experiment/result relationships.	A parameter change can be traced through actual code objects to experiment/result artifacts.
P1	server/engine/paper_parser.py:242-307 + parser boundary	Add real PDF/DOCX adapters and preserve page/paragraph/table locations and extraction status.	Uploaded PDF/DOCX produces a structured research-object model with explicit extraction success/degraded/failure.
P1	client/src/components/BlastRadiusHeader.jsx:114-120	Remove Blast Radius Score and percentage; show impact status and evidence summary.	No researcher-facing scientific-looking 0-100 impact percentage.
P1	client/src/App.jsx:270-386	Move AST graph/agent trace/technical lineage into developer mode; redesign normal IA around Overview/Changes/Paper Impact/Experiments/Issues.	A researcher can use the product without seeing AST/agent architecture.
P1	client/src/components/WhatIfConsole.jsx:132-157	Rename Blast Radius action and make result categories explicit: Verified/Likely/Needs review/No dependency found/Unable to verify/Analysis failed.	What-if answers report evidence state, not a score.
P1	server/routers/pr_router.py:12-106	Connect PR to real Git branch/commit, run diff-based impact analysis, persist findings and provide evidence-backed merge checklist.	Opening a Research PR creates a real analysis against the proposed commit.
P1	server/engine/jobs.py:1-30	Persist job records and execute real analysis stages; add partial/failure state and progress events.	Jobs survive restarts and report genuine progress.
P1	cli.py:21-28	Remove mock content/diff. Parse supplied diff or PR link and submit real repository/project state.	CLI never prints or submits "Mocked files payload" / "mock content".
P2	server/tests/*.py	Fix test environment/imports, then add dependency, evidence, mutation, false-positive, false-negative, security and E2E tests.	CI demonstrates measurable precision/recall and failure behavior on controlled fixtures.
P2	repository root	Add missing OSS release artifacts: license, CODE_OF_CONDUCT.md, CHANGELOG.md, issue/PR templates, runnable fixtures.	Repository satisfies PDF's open-source release checklist.

12. Recommended Target Runtime Architecture

The implementation should converge on the following runtime path. This is the smallest architecture that matches the PDF without turning PaperBlast into a generic research platform.

Stage	Required behavior	Current code to replace/extend
1. Project state	Load persistent ResearchProject, repository commit and paper version.	server/engine/persistence/db_adapter.py; server/domain/models.py
2. Change	Retrieve real branch/commit/diff or parse proposed what-if intent.	server/routers/pr_router.py; Git connector; impact_engine.py
3. Code parse	AST parse supported Python; explicit failure status; exact locations.	server/engine/code_parser.py
4. Manuscript parse	Real PDF/DOCX/LaTeX adapters; explicit extraction status and source locations.	server/engine/paper_parser.py; server/main.py
5. Deterministic graph	Build calls, references, assignments, parameter flow, experiments, outputs and paper-object links.	server/engine/graph_builder.py + new dependency analyzer
6. AI semantic layer	Resolve ambiguous natural language to candidate artifact IDs; never author evidence.	server/engine/impact_engine.py + groq_client.py
7. Evidence	Store source artifact, location, extracted text/value, relationship, verification status.	server/domain/models.py + factories.py
8. Verification	Reject any finding whose edges/evidence cannot be resolved deterministically.	new verifier module or server/engine/status_model.py extension
9. Impact result	Return explicit statuses and evidence-backed findings; exact numerical changes remain unknown until execution.	server/engine/impact_engine.py
10. Research PR	Attach result to real commit/diff; comments/reviews/merge checklist persisted.	server/routers/pr_router.py + collaboration/persistence
11. UI	Show simple research workspace and evidence panel; developer mode contains technical diagnostics.	client/src/App.jsx + components

North-star acceptance test: Give PaperBlast a real repository, make a real parameter change, and verify that the system can trace the change through actual code/configuration relationships to experiments/results/paper artifacts; every important finding must open to source evidence; if the relationship cannot be proven, the system must say so.

13. Testing & Verification Gaps

The PDF requires more than conventional unit tests: it explicitly calls for dependency tests, evidence tests, integration tests, mutation tests, false-positive/false-negative tests, regression/security tests and end-to-end analysis-to-Research-PR tests. (PDF p. 19.)

Observed test execution result. Running `python -m pytest server/tests/ -q` against the uploaded snapshot fails during test collection because the environment lacks the `groq` package. Four modules fail to import: `test_agent.py`, `test_impact_engine.py`, `test_provenance_graph.py` and `test_skeptic.py`. This is not a claim that the project's `requirements.txt` is missing `groq` — it contains it — but it is evidence that the repository's test suite is not self-contained/runnable in the audited environment without dependency installation.

Required test layer (PDF)	Current evidence	Gap
Parser unit tests	<code>server/tests/test_code_parser.py</code>	Exists; should add malformed-AST failure semantics and source-location fixtures.
Dependency tests	No dedicated dependency/data-flow suite identified.	Missing dedicated call/reference/assignment/data-flow assertions.
Evidence tests	No dedicated comprehensive evidence verification suite identified.	Missing proof that every VERIFIED finding resolves to real source evidence.
Integration tests	Not evidenced as full real-project → API/UI workflow.	Missing.
Mutation tests	No mutation suite identified.	Missing.
False-positive tests	No dedicated harmless-change suite identified.	Missing.
False-negative tests	No dedicated known-dependency suite identified.	Missing.
Regression tests	No structured defect regression suite identified.	Missing.
Security tests	Security code/docs exist; automated repository-ingestion security suite not evidenced.	Partial.
E2E	No clean-checkout → ingest → analyze → evidence → Research PR suite identified.	Missing.

Recommended first benchmark fixture: create a tiny research project with `config` → `training function` → `experiment runner` → `CSV result` → `table` → `claim`. Mutate the `config` parameter and assert the exact expected artifact chain. Then create harmless mutations and assert they are not flagged. This directly exercises the product moat described in the PDF.

14. Production Definition of Done — Current Gap Checklist

PDF Definition of Done item (p. 23)	Current assessment	Evidence / remediation
Clean checkout can be installed/run	PARTIAL	requirements/package files exist; test collection fails without installed groq; deployment entry mismatch must be fixed.
Researcher can analyze a real supported project	PARTIAL	Direct upload/GitHub ingestion exists, but project state is not persisted and analysis path remains heuristic.
Real what-if question gets evidence-backed assessment	PARTIAL	What-if UI exists; AI still creates key semantic/factual result structures.
Affected / no-dependency / unable-to-verify states separate	IMPLEMENTED / PARTIAL	VerificationStatus exists; no-dependency/failed semantics need stricter runtime gating.
Every important finding has navigable evidence	PARTIAL	EvidencePanel exists; AI-generated lineage must be removed as factual evidence.
No static mocks/simulation findings	FAIL	server/main.py:40–41 simulated job; cli.py:23–28 mock payload.
No silent fallback	FAIL	code_parser.py:135–139 regex fallback; impact engine static fallback path.
Failures visible/actionable	PARTIAL	Statuses exist, but failed analysis still carries lineage-like output.
Version-aware; stale analysis cannot appear current	PARTIAL	AnalysisVersion exists, but persistent version state is missing.
AI semantic role only	FAIL	AI returns affected sections and lineage; prompt still requests impact score.
Parsers covered by automated tests	PARTIAL	Tests exist but suite collection fails in audited environment.
Mutation tests demonstrate change detection	FAIL	Not evidenced.
False-positive restraint	FAIL	Not evidenced.
Research PRs connect commits to findings/review	PARTIAL	PR/review APIs exist, but real commit/diff analysis attachment is not wired.
GitHub integration works against real repos	PARTIAL	Ingestion exists; deployment/runtime and versioned diff workflow are incomplete.
Security controls protect ingestion/execution	PARTIAL	Ingestion has limits; sandboxed execution worker is absent.
Self-hosting/deployment documented/reproducible	PARTIAL/FAIL	Docs exist; Vercel runtime wiring conflicts with FastAPI tree.
OSS contribution workflow complete	PARTIAL	Several docs exist; release artifacts/fixtures are incomplete.
Public benchmark reports actual performance	FAIL	BENCHMARKS.md contains target-style claims and explicitly says AI cost is not measured / models mocked locally.
Researcher UI remains simple	FAIL	AST/agent/blast-radius concepts remain prominent.

15. Prioritized Implementation Roadmap

Phase	Work	Why first	Definition of complete
P0 — Trust & runtime	Remove simulated jobs, mock CLI payloads, parser regex fallback, AI lineage authority, static fallback findings; fix deployment entry point.	These are direct violations of the PDF's production/no-fabrication contract.	No required failure path can manufacture a plausible impact result; production API runs the real backend.
P0 — Deterministic engine	Implement real AST reference/call/assignment/data-flow graph and experiment/result mapping.	This is the product's scientific moat and the basis for evidence.	A controlled parameter mutation produces a deterministic artifact chain.
P0 — Persistence	Postgres/Supabase adapter + ResearchProject/versions/findings/evidence/PR/job persistence.	Without persistent state, version-aware research workflows cannot be real.	Restarting service does not erase project state or analysis records.
P1 — Manuscript ingestion	Real PDF/DOCX/LaTeX adapters with extraction status and exact locations.	The current frontend promises document support the Python runtime does not truly implement.	Supported documents produce inspectable research objects.
P1 — Verification	Strict evidence verifier; VERIFIED only from deterministic support; AI limited to semantic candidates/explanations.	Required to satisfy trust model.	Every VERIFIED finding is traceable to deterministic evidence.
P1 — Research PR	Real Git branch/commit/diff → impact analysis → comments/review/checklist.	Collaboration is a supporting mechanism, not the core engine.	PR findings are linked to real commits and persist across sessions.
P1 — UI redesign	Overview / Changes / Paper Impact / Experiments / Issues; developer mode for technical diagnostics.	PDF explicitly requires researcher-first UX.	Normal researcher workflow contains no AST/agent/blast-radius jargon.
P2 — Quality benchmark	Mutation + FP/FN + integration + security + E2E; publish precision/recall and failure metrics.	The PDF requires measurable accuracy rather than marketing claims.	CI and public benchmark show actual performance and limitations.

16. Detailed Code Evidence Appendix

Selected exact source excerpts are included below to make the audit independently actionable. These excerpts are taken from the uploaded ZIP snapshot.

A. AST fallback that must be removed

```
128: def parse_python_ast(code: str, file_path: str) -> List[Dict[str, Any]]:
129:     lines = code.split('\n')
130:     try:
131:         tree = ast.parse(code, filename=file_path)
132:         visitor = PythonCodeVisitor(file_path, lines)
133:         visitor.visit(tree)
134:         return visitor.symbols
135:     except SyntaxError as e:
136:         logger.warn(f"SyntaxError parsing Python file {file_path}", {"error": str(e)})
137:         # Return DEGRADED_EXTRACTION via a special symbol or fallback to regex
138:         # For resilience, we run the legacy regex fallback if AST completely fails
139:         return parse_python_regex_fallback(code, file_path, extraction_status="PARTIAL")
140:     except Exception as e:
141:         logger.error(f"Unexpected AST error on {file_path}", {"error": str(e)})
142:     return []
```

Why it matters: the PDF requires explicit failure/uncertainty rather than converting a failed parse into a plausible symbol set (PDF pp. 7-8, 14).

B. Simulated analysis job

```
33: @app.post("/api/jobs/analyze")
34: async def start_analysis_job(req: AnalyzeRequest, background_tasks: BackgroundTasks):
35:     job_id = create_job()
36:
37:     async def process_job():
38:         update_job_status(job_id, JobStatus.PROCESSING)
39:         try:
40:             await asyncio.sleep(1) # simulate work
41:             update_job_status(job_id, JobStatus.READY, result={"message": "Analysis Complete"})
42:         except Exception as e:
43:             update_job_status(job_id, JobStatus.FAILED, error=str(e))
44:
45:     background_tasks.add_task(process_job)
46:     return {"jobId": job_id}
```

Why it matters: this is a direct no-mock violation. A job should execute the real pipeline and only transition to READY after real processing (PDF p. 14).

C. In-memory persistence

```
24: class InMemoryAdapter(DatabaseProvider):
25:     """
26:     Development fallback for Vercel Dev.
27:     In production, this must be swapped for Postgres (e.g. Vercel Postgres / Supabase).
28:     """
29:     def __init__(self):
30:         self.users = {}
31:         self.memberships = []
32:         self.invitations = {}
33:         self.conversations = {}
34:         self.projects = set()
35:         self.audits = []
36:
37:
38: # Factory logic
39: def get_db_provider() -> DatabaseProvider:
40:     # If DATABASE_URL is set, we could return a PostgresAdapter
41:     # If os.getenv('DATABASE_URL'):
42:     #     return PostgresAdapter(os.getenv('DATABASE_URL'))
43:     if not hasattr(get_db_provider, '_instance'):
44:         get_db_provider._instance = InMemoryAdapter()
45:     return get_db_provider._instance
```

Why it matters: the PDF requires persistent project/version state and explicitly places persistence in the core backend architecture (PDF pp. 8, 11-12).

D. Mock CLI payload

```
21: def analyze_cmd(args):
22:     url = f"{API_URL}/jobs/analyze"
23:     print(f"Starting analysis for PR Diff... (Mocked files payload)")
24:     payload = {
25:         "files": [],
26:         "paper": {"content": "mock content"},
27:         "changeQuery": "mock diff"
28:     }
29:     try:
30:         r = httpx.post(url, json=payload, timeout=10.0)
```

Why it matters: the PDF's production contract prohibits static/demo-only inputs and simulated results (PDF pp. 7, 14).

E. Researcher-facing Blast Radius Score

```
114: <div className="flex flex-col items-center border-r-2 border-black pr-5">
115:   <span className="text-slate-700 font-bold uppercase text-[10px] tracking-wider">Blast Radius Score</span>
116:   <div className="flex items-center gap-2 mt-0.5">
117:     <span className="text-xl font-extrabold text-black">{score !== null ? `${score}%` : '-'}</span>
118:     <span className={`badge ${getRiskClass(risk)}`}>{risk}</span>
119:   </div>
120: </div>
```

Why it matters: the PDF explicitly removes Blast Radius Score and uncalibrated confidence percentages from the researcher-facing product (PDF p. 7; pp. 12-13).

F. Vercel routing vs actual backend

```
1: {
2:   "version": 2,
3:   "buildCommand": "npm run build",
4:   "outputDirectory": "dist",
5:   "rewrites": [
6:     {
7:       "source": "/api/(.*)",
8:       "destination": "/api/index.js"
9:     },
10:    {
11:      "source": "/(.*)",
12:      "destination": "/index.html"
13:    }
14:  ]
15: }
```

```
1: import app from '../server/index.js';
```

Why it matters: the repository's actual FastAPI app is server/main.py:31, while the deployment adapter imports a missing server/index.js. This must be reconciled before production deployment.

17. Final Verdict

The updated codebase is not a failed implementation; it contains substantial work toward the target architecture. The strongest improvements are the AST parser, domain/evidence models, explicit verification statuses, Research PR/auth structures, and the beginning of provenance-aware UI.

Nevertheless, the codebase is not yet production-compliant with the supplied PDF. The most serious gaps are in the exact areas the PDF treats as non-negotiable: no fabricated/silent fallback behavior, deterministic-first verification, real project/version state, real change/diff analysis, and a simple researcher-facing product.

Release recommendation: NO-GO at the current snapshot. Do not position this repository as satisfying the final production blueprint until the P0 trust/runtime issues are closed and the P1 workflow is demonstrably exercised end-to-end.

The clearest next milestone is a single vertical slice: real repository + real paper → real commit diff → deterministic AST/dependency traversal → evidence-backed finding → Research PR → review → persistent versioned state. Once that slice is truthful and testable, the remaining UI, OSS and benchmark work can be completed around it.

Specification references used in this audit: PDF pp. 2–4 (product definition), p. 8 (modifications), p. 9 (additions), pp. 11–13 (architecture/data model/statuses), p. 14 (no-mock production contract), p. 15 (AI boundaries), p. 16 (collaboration), p. 17 (OSS), p. 18 (security), p. 19 (testing), pp. 20–21 (implementation phases), pp. 23–24 (definition of done/scope), p. 26 (final blueprint). Audited source: Research_Blast_Radius-main.zip, uploaded in this conversation. Exact code line references in this report refer to that snapshot.